

1. Introduction

Training very deep convolutional networks poses three well-known difficulties: vanishing gradients in backward propagation, diminishing feature reuse in forward propagation, and prohibitively long training time. Huang et al. [1] address all three by randomly dropping entire residual blocks during training, bypassing them with identity skip connections. Each block ℓ is assigned a survival probability under a linear decay schedule,

$$p_\ell = 1 - \frac{\ell}{L}(1 - p_L), \quad (1)$$

so earlier, low-level layers survive more often. A Bernoulli gate $b_\ell \sim \text{Bernoulli}(p_\ell)$ determines whether block ℓ fires each mini-batch, yielding the update $H_\ell = \text{ReLU}(b_\ell f_\ell(H_{\ell-1}) + \text{id}(H_{\ell-1}))$; at test time all blocks are active and residual outputs are scaled by p_ℓ . On a 110-layer ResNet trained on CIFAR-10, this reduces test error from 6.41% to 5.25% and cuts training time by 25%.

2. Chosen Result

The primary target is **Figure 3 (left)** of the paper: per-epoch test error and training loss curves for constant-depth and stochastic-depth 110-layer ResNets on CIFAR-10 over 500 epochs. This is the paper’s central empirical claim, showing stochastic depth achieves lower final test error and converges the training loss faster. The secondary target is **Figure 7**: mean gradient magnitude of the first convolutional layer throughout training. This figure provides a mechanistic explanation for the performance gain—stochastic depth shortens the expected backpropagation path, keeping gradients healthier in early layers and explaining why the network continues to improve even after the second learning-rate drop. All figures and tables are in the Appendix.

3. Methodology

Architecture. The same 110-layer ResNet ($6N+2$, $N=18$): three groups of 18 residual blocks with 16/32/64 filters, a Conv–BN–ReLU stem, global average pool, and linear classifier.

Differences from the authors’ Torch 7 code. (1) *Skip-connection projection*: the original uses average pooling + zero-channel padding at transition blocks (Option A of He et al. [2]); the re-implementation uses a 1×1 strided convolution + batch normalization (Option B), a learnable projection with slightly more capacity. (2) *Gate placement*: the original training loop sets a `gate` boolean on each module before every mini-batch. The re-implementation moves the Bernoulli draw inside `StochasticBlock.forward()`, eliminating the two-place coupling with no behavioral change.

Training. SGD with Nesterov momentum (0.9), weight decay 10^{-4} , learning rate = 0.1 divided by 10 at epochs 250 and 375, batch size 128, 500 epochs, 45k/5k train/validation split, standard CIFAR-10 augmentation. Trained on an NVIDIA A100 in Google Colab (authors used a TITAN X).

4. Results & Analysis

Quantitative comparison. Both models match or exceed the paper (Table 1). Figures 2 and 4 are faithful reproductions; the originals are shown alongside for comparison (Figures 1 and 3). Stochastic depth achieves lower error throughout training and continues improving after the second learning-rate drop, while the constant-depth baseline approximately plateaus. The baseline’s gradient magnitude collapses to near zero after epoch 250 and stays there, while stochastic depth remains elevated through epoch 500, consistent with the paper’s vanishing-gradient explanation.

The re-implementation’s baseline (5.81%) outperforms the paper’s (6.41%) by a larger margin than the stochastic model does (5.15% vs. 5.25%). Three factors likely contribute: Option B skip connections are strictly more expressive than Option A; newer cuDNN kernels on the A100 affect floating-point scheduling; and the authors themselves note in their README that “it is normal to get a $\pm 0.2\%$ difference from our reported results on CIFAR-10.”

Error analysis by class. Table 2 compares per-class errors and key pairwise confusions on the full 10,000-image test set. Gains concentrate in semantically ambiguous categories: the four largest improvements are truck (-2.2%), dog (-2.0%), cat (-1.9%), and horse (-1.3%), while stochastic depth is slightly *worse* on unambiguous classes like ship ($+0.8\%$) and automobile ($+0.3\%$). Pairwise confusion confirms this: cat $\rightarrow$ dog drops from 6.0% to 5.5%, dog $\rightarrow$ cat from 6.1% to 5.2%, and truck $\rightarrow$ automobile from 2.6% to 1.6%. This is consistent with the implicit ensemble interpretation—averaging over random subnetworks reduces variance most on the hard within-group boundaries where a single deterministic network struggles most.

5. Reflections

Lessons learned. The most instructive part of the re-implementation was translating the Torch 7 gate mechanism into PyTorch. The original design sets gate state externally from the training loop and reads it inside the module’s forward pass—a two-place coupling that is easy to break. Moving the Bernoulli draw inside `StochasticBlock.forward()` eliminates this coupling entirely with no behavioral difference. The Option A vs. Option B difference is a reminder that implementation choices which appear minor can shift final accuracy by several tenths of a percent.

The class-level confusion analysis was an unplanned addition that turned out to be the most interesting result. The pattern—large gains on ambiguous animal and vehicle pairs (cat/dog, truck/automobile, horse/deer), small losses on visually distinctive classes (ship, airplane)—was not predicted in advance and suggests the ensemble effect of stochastic depth operates selectively on hard decision boundaries rather than providing a uniform accuracy lift.

Future directions. Measuring prediction variance across multiple stochastic forward passes for ambiguous vs. unambiguous classes would directly test whether variance reduction drives the selective gains. Reproducing Figure 8 (p_L sensitivity and depth $\times p_L$ heatmap) would complete the re-implementation and confirm the linear decay schedule’s advantage over uniform survival probabilities.

References

- [1] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K.Q. Weinberger. Deep networks with stochastic depth. In *ECCV*, 2016. arXiv:1603.09382.
- [2] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016. arXiv:1512.03385.

Appendix: Tables and Figures

Table 1: CIFAR-10 test error.

Model	Paper	Re-impl.
Constant depth	6.41%	5.81%
Stochastic depth	5.25%	5.15%
Relative improv. ↓	18.1%	15.5%

Table 2: Per-class test error (%). Negative Δ favours stochastic depth.

Class	Base	Stoch	Δ
airplane	5.3	5.4	+0.1
automobile	2.1	2.4	+0.3
bird	9.2	9.4	+0.2
cat	12.3	10.4	−1.9
deer	4.1	4.0	−0.1
dog	10.0	8.0	−2.0
frog	4.2	3.3	−0.9
horse	4.7	3.4	−1.3
ship	2.8	3.6	+0.8
truck	5.1	2.9	−2.2

Table 3: Selected pairwise confusion rates (%). Negative Δ favours stochastic depth.

Confusion	Base	Stoch	Δ
cat $\rightarrow$ dog	6.0	5.5	−0.5
dog $\rightarrow$ cat	6.1	5.2	−0.9
truck $\rightarrow$ auto	2.6	1.6	−1.0
auto $\rightarrow$ truck	1.6	1.9	+0.3

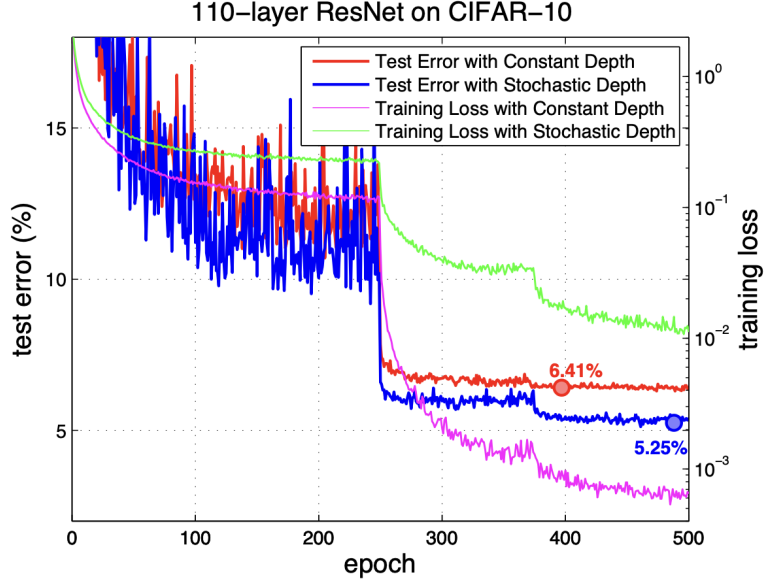


Figure 1: Figure 3 (left) from Huang et al. [1]: test error and training loss for the 110-layer ResNet on CIFAR-10. Constant depth achieves 6.41%; stochastic depth achieves 5.25%.

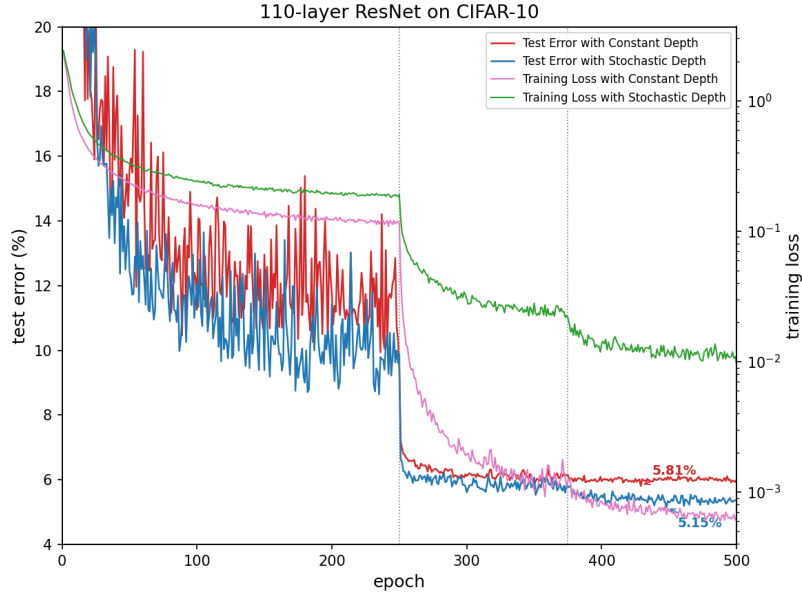


Figure 2: Reproduced Figure 3: test error (left axis) and training loss (right axis, log scale) over 500 epochs. Constant depth achieves 5.81%; stochastic depth achieves 5.15%. Dotted lines mark learning rate drops at epochs 250 and 375.

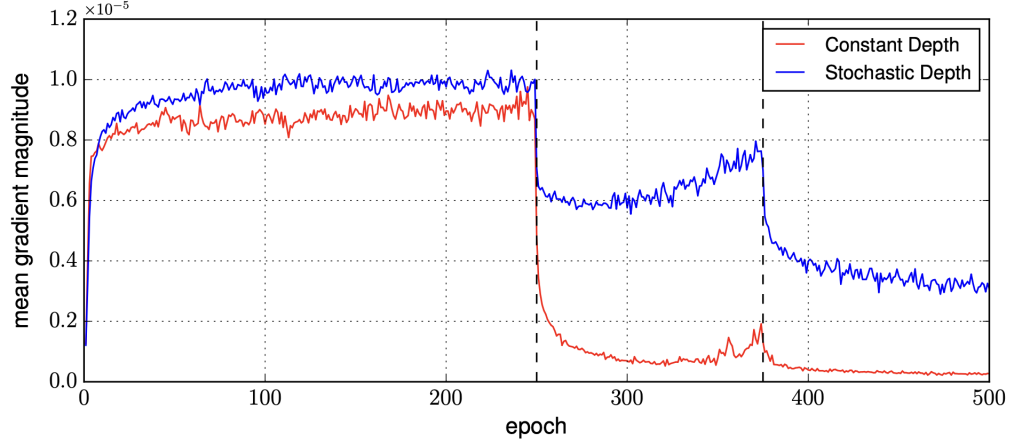


Figure 3: Figure 7 from Huang et al. [1]: mean gradient magnitude of the first convolutional layer. After each learning rate drop the constant-depth gradient shrinks sharply while stochastic depth remains elevated.

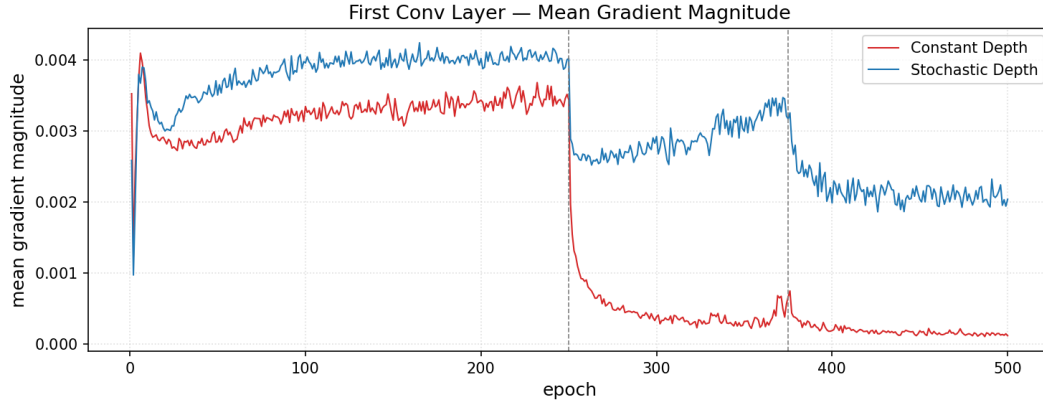


Figure 4: Reproduced Figure 7: mean gradient magnitude of the first conv layer. After epoch 250 the baseline collapses to near zero while stochastic depth maintains healthy gradients through epoch 500.